

计组复习

summarized by よおんはく

PS:

不考嵌入式、不考第九章、
没有模型题
总结的缩写记得看
流水线也砍掉了

英文缩写名词解释

~~CISC~~ ~~RISC~~: Complex Instruction Set Computer 精简指令集计算机
~~RISC~~ ~~CISC~~: Reduced Instruction Set Computer 复杂指令集计算机

英文缩写	英文全称	中文全称
RAM	Random Access Memory	随机存取存储器
ROM	Read-Only Memory	只读存储器
SRAM	Static Random Access Memory	静态随机存取存储器
DRAM	Dynamic Random Access Memory	动态随机存取存储器
CAM	Content-Addressable Memory	内容寻址存储器
FIFO	First-in First-out	先入先出策略
LRU	Least Recently Used	最近最少访问
EAT	Effective Access Time	有效访问时间

英文缩写	英文全称	中文全称
EAT	Effective Access Time	有效访问时间
TLB	Translation Lookaside Buffer	转换后备缓冲器
MMU	Memory Management Unit	内存管理单元

英文缩写	英文全称	中文全称
I/O	Input/Output	输入输出
DMA	Direct Memory Access	直接内存访问
RDMA	Remote Direct Memory Access	远程直接内存访问
HDD	Hard Disk Drive	硬盘
SSD	Solid State Drive	固态硬盘
RAID	Redundant Array of Independent Disks	独立磁盘冗余阵列
RS	Reed-Solomon	里德-所罗门编码

英文缩写	英文全称	中文全称
VM	Virtual Machine	虚拟机
LPAR	Logical Partition	逻辑分区
PV	Paravirtualization	半虚拟化
HVM	Hardware Virtual Machine	硬件虚拟化/全虚拟化
KVM	Kernel-based Virtual Machine	基于内核的虚拟机
VMX	Virtual Machine Extension	虚拟机扩展指令

英文缩写	英文全称	中文全称
SoC	System-on-Chip	片上系统
PLD	Programmable Logic Device	可编程逻辑设备
PLA	Programmable Logic Array	可编程逻辑阵列
PAL	Programmable Array Logic	可编程阵列逻辑
FPGA	Field Programmable Gate Array	现场可编程逻辑门阵列
ASIC	Application-specific Integrated Circuit	专用集成电路
VHDL	Very-High-Speed Integrated Circuit Hardware Description Language	高速集成电路硬件描述语言

第一章

云计算

- 云计算出现的契机：网络技术的发展、大型数据中心的兴起：冗余计算资源
- 云计算的优势：硬件成本、维护成本、管理成本大幅下降；服务弹性；资源利用率
- 常见的云计算类型：计算即服务、软件即服务、平台即服务、基础设施即服务

冯诺依曼体系架构

冯·诺依曼结构也称普林斯顿结构，是一种将程序指令存储器和数据存储器合并在一起的存储器结构。程序的执行过程称为“取-译码-执行”

主要特点有：

- (1) 在CPU和主存之间有且仅有一个数据通路（也被称为冯-诺依曼瓶颈）
- (2) 由三部分硬件系统构成：中央处理单元（CPU）、主存、输入/输出系统
- (3) 可以执行指令序列；
- (4) 数据以二进制表示；
- (5) 指令由操作码和操作数组成；

第二章

进制转换

- 重复减法
- 把 190_{10} 转换成基数为 3 的数

$$190_{10} = 21001_3$$

$$\begin{array}{rcl}
 190 & & \\
 - 162 & = 3^4 \times 2 & \\
 \hline
 28 & & \\
 - 27 & = 3^3 \times 1 & \\
 \hline
 1 & & \\
 - 0 & = 3^2 \times 0 & \\
 \hline
 1 & & \\
 - 0 & = 3^1 \times 0 & \\
 \hline
 1 & & \\
 - 1 & = 3^0 \times 1 & \\
 \hline
 0 & &
 \end{array}$$

- 把 0.8125_{10} 转换成基数为 2 的数

$$0.8125_{10} = 0.1101_2$$

$$\begin{array}{r}
 0.8125 \\
 - 0.5000 = 2^{-1} \times 1 \\
 \hline
 0.3125 \\
 - 0.2500 = 2^{-2} \times 1 \\
 \hline
 0.0625 \\
 - 0 = 2^{-3} \times 0 \\
 \hline
 0.0625 \\
 - 0.0625 = 2^{-4} \times 1 \\
 \hline
 0
 \end{array}$$

- 把 11010100011011_2 转换成八进制 ($8=2^3$) 的数

011	010	100	011	011
3	2	4	3	3

布斯算法

- 布斯算法

- 如果当前的乘数位是 1，前面的乘数位是 0，则从乘积中减去被乘数
- 如果当前的乘数位是 0，前面的乘数位是 1，我们就把被乘数加到乘积上
- 如果我们有一个 00 或 11 对，我们只是移位。
- 假设一个虚构的 “0” 开头
- 每步移位

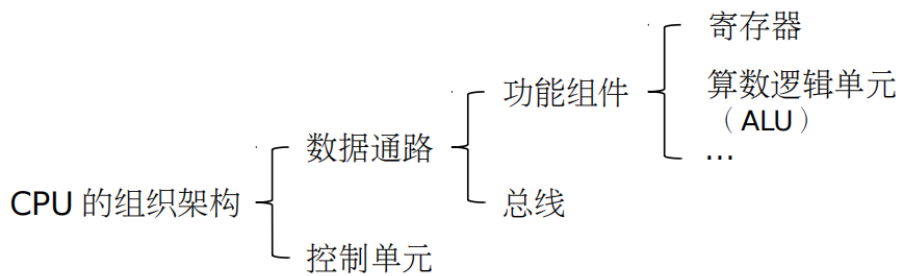
0011		
<u>x 0110</u>		
+	0000	(shift)
-	0011	(subtract)
+	0000	(shift)
+	0011	(add)
00010010		
We see that $3 \times 6 = 18$!		

第三、四章

费林分类几个缩写应该知道

SISD(单指令流单数据流)、SIMD(单指令流多数据流)、
MISD(多指令流单数据流)、MIMD(多指令流多数据流)

cpu基本知识和组织架构

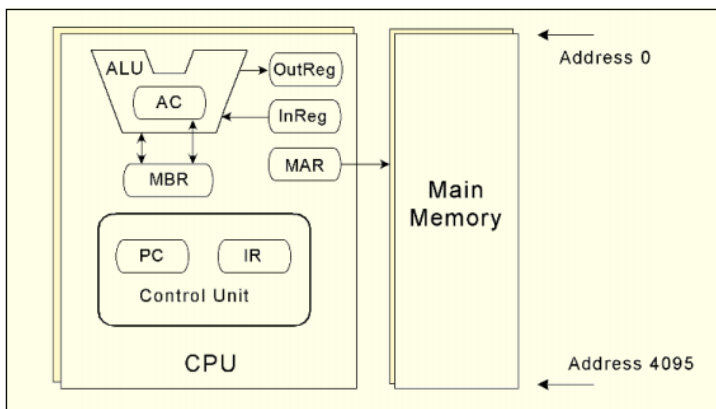


- 数据通路（Datapath）
 - 寄存器（Register）：CPU 内部的数据存储单元
 - 算数逻辑单元（ALU）：执行逻辑运算和算术运算
 - 总线（Bus）：一组连线的集合，CPU 通过总线与其他组件进行通信
- 控制单元（Control Unit） 监视、调度、控制数据通路中的数据传输

循环冗余检验CRC

计网学了，略

marie! ! 重要



七个寄存器，分别多少位，分别什么作用

1. 累加寄存器（AC）~~代码为大数据块提供错误检测~~：16 位；用来存储指令运算操作数和运算结果

2. 存储器地址寄存器（MAR）：12 位，保存一条指令的内存地址或一条指令的操作数
3. 存储器缓冲寄存器（MBR）：16位，用于在数据从内存中检索或放置之前保存数据
4. 程序计数器（PC）：12 位，保存下一条要执行的程序指令的地址
5. 指令寄存器（IR）：16 位寄存器，保存将要执行的下一条指令
6. 输入寄存器（InReg）：8位，保存来自输入设备的数据
7. 输出寄存器（OutReg）：8 位寄存器，保存要输出到输出设备的数据

marie的指令集

16 位指令： 4 位操作码（ Opcode ） + 12 位地址（ Address ）



基本指令、扩展指令

指令编码	指令	含义
0x0	JNS x	将 PC 写入地址为 x 的存储单元，并将 x+1 写入 PC
0x1	LOAD x	将地址为 x 的存储单元中的值存入 AC
0x2	STORE x	将 AC 中的值存储到地址为 x 的存储单元中
0x3	ADD x	将地址 x 中的值与 AC 中的值相加的结果存入 AC
0x4	SUBT x	将 AC 中的值减去地址 x 中的值的结果存入 AC
0x5	INPUT	从键盘输入一个数值到 AC
0x6	OUTPUT	将 AC 中的数值输出到显示器
0x7	HALT	终止程序的运行
0x8	SKIPCOND	有条件地跳过下一条指令
0x9	JUMP x	将 x 的值装入 PC
0xa	CLEAR	将 AC 中的值置为 0
0xb	ADDI x	读取地址 x 中的地址，将其对应的值与 AC 中的值相加的结果存入 AC
0xc	JUMPI x	读取地址 x 中的地址，将其对应的值写入 PC
0xd	LOADI x	读取地址 x 中的地址，将其对应的值写入 AC
0xe	STOREI x	读取地址 x 中的地址，将 AC 写入其对应的存储单元

其中 0x1–0x9 为基本 指令，
0x0 、 0xa– 0xe 为扩展指令。

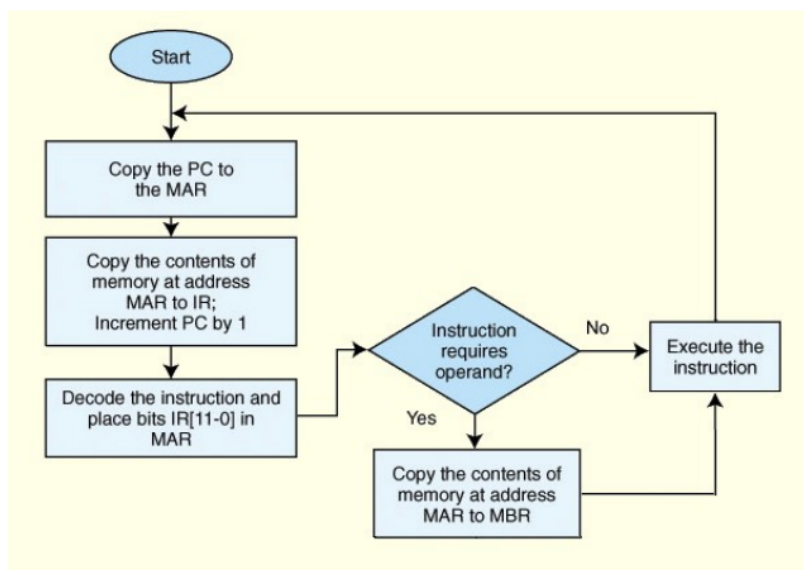
rtn是什么？什么作用就是marie的一个最基本的微操作

寄存器传输表示 (opRegister Transfer Notation, RTN) 或者寄存器传输语言 (opRegister Transfer Language, RTL) 来形式化地描述marie的最基本的微操作。

LOAD x	MAR ← x MBR ← M[MAR] AC ← MBR	ADD x	MAR ← x MBR ← M[MAR] AC ← AC + MBR	LOADI x	MAR ← x MBR ← M[MAR] MAR ← MBR MBR ← M[MAR] AC ← MBR
--------	-------------------------------------	-------	--	---------	--

了解marie架构指令执行的具体过程

- 取指 - 译码 - 执行周期
- 使用 RTN 描述 MARIE 的指令执行过程
 - $MAR \leftarrow PC$
 - $IR \leftarrow M[MAR], PC \leftarrow PC + 1$
 - $MAR \leftarrow IR[11-0],$
 $OP \leftarrow DECODE(IR[15-12])$
 - $MAB \leftarrow M[MAR],$
 $EXEC(OP)$



控制信号砍掉了，不考

无

实际的体系结构，47页ppt重点

- 按照指令集分类
 - 复杂指令集计算机（Complex Instruction Set Computer, CISC）：
 - 指令数量多、长度各异
 - ❑ CISC 代表：Intel 体系结构（x86 体系结构）
 - 精简指令集计算机（Reduced Instruction Set Computer, RISC）：
 - 指令数量少，所有指令长度相同
 - ❑ RISC 代表：MIPS、ARM

第五章

指令集架构

- 指令集架构（ISA）是计算机的抽象模型，是软件和硬件之间的接口
- ISA 包括了一个计算机体系结构中所有指令构成的集合，以及这些指令的编码方法
- 指令集需要和计算机体系结构相匹配
- 衡量指令集标准：程序占用空间的大小、指令集的复杂性、指令的长度、指令集中的指令总数

- 设计指令集时，要考虑：指令长度、操作数op的数量、可寻址寄存器的数目、内存组织（字节或字寻址）、寻址方式
- 短指令通常比长指令好：读取编译指令更快、限制了指令的数量、对操作数个数和大小有更严格的要求
- 定长指令译码相对容易但会浪费存储空间
- 通过允许操作数字段位数可变（扩展操作码）能增加定长指令中可表示的操作数个数
- 现有体系结构通常是字节编址的

大小端，分别是什么，各自的特点，什么场景使用哪个，方式，各自的优势

number 0x12345678：（不是数字位顺序，而是字节顺序）

Address 	00	01	10	11
Big Endian	12	34	56	78
Little Endian	78	56	34	12

Big endian:

- Ø更自然
- Ø数字的符号可以通过查看地址偏移0处的字节来确定
- Ø字符串和整数以相同的顺序存储
- Ø计算机网络采用大端

Little endian:

- Ø使得在非字边界时放置值变得更容易
- Ø从16位整数地址到32位整数地址的转换不需要任何算术
- Ø从高位数转换为低位数时寻址更方便

cpu内部的存储

有三种选择：堆栈型架构、累加器型架构、通用寄存器

操作数、三种选择的不同在于寄存器不一样

存储方式 指令格式	堆栈型	累加器型	通用寄存器型
零地址	✓	✓	✓
单地址	✓	✓	✓
双地址	✗	✗	✓
三地址	✗	✗	✓

堆栈型架构、累加器型架构、通用寄存器型架构

指令集结构类型	优点	缺点
堆栈型	表示计算的简单模型；指令短小	堆栈不能被随机访问，从而很难生成有效代码；同时，由于堆栈是瓶颈，所以很难被高效地实现
累加器型	减小了机器的内部复杂度；指令短小	由于累加器是唯一的暂存器，这种机器的存储器通信开销最大
通用寄存器型	是代码生成最常见的模型	所有操作数均需命名，且显式表示，指令较长
*存储器-存储器型	是一种最紧密的编码方式，无需“浪费”寄存器保存变量	指令字长多种多样。每条指令的执行时钟周期数也大不一样，对存储器的频繁访问将导致存储器访问瓶颈问题
*寄存器-存储器型	可以直接对存储器操作数进行访问，容易对指令进行编码，且其目标代码较小	指令中的操作数类型不同。在一条指令中同时对一个寄存器操作数和存储器操作数进行编码，将限制指令所能够表示的寄存器个数。由于指令的操作数可以存储在不同类型的存储器单元，所以每条指令的执行时钟周期数也不尽相同
*取-存型	简单，指令字长固定，是一种简单的代码生成模型，各种指令的执行时钟周期数相近	和指令中含有对存储器操作数访问的结构相比，指令条数多，因而其目标代码较大

逆波兰式（比期中的简单，但是有坑）

后缀表示法（postfix notation）操作符放在操作数之后，

即逆波兰式（reverse Polish notation, RPN）例如： $Z = XY +$

扩展操作码，变化的是操作码，指令长度不变；转义符占不占地址？不占。

当使用固定长度的指令时，不需要操作数的操作（如HALT）必然会浪费一些空间，恢复这些空间的一种方法是使用扩展操作码

扩展操作码（Expanding Opcodes）在定长的指令中，通过增加变长的操作码来表示更多的指令

```

0000 R1   R2   R3   }
...           } 15 three-address codes
1110 R1   R2   R3   }
1111 - escape opcode

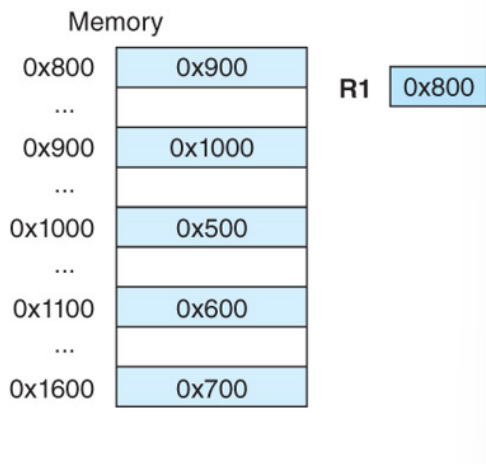
1111 0000 R1   R2   }
...           } 14 two-address codes
1111 1101 R1   R2   }
1111 1110 - escape opcode

1111 1110 0000 R1   }
...           } 31 one-address codes
1111 1111 1110 R1   }
1111 1111 1111 - escape opcode

1111 1111 1111 0000 }
...           } 16 zero-address codes
1111 1111 1111 1111 }

```

寻址方式、常见的几种寻址方式，要会才行。



Ø立即寻址：LOAD 0x800，立即寻址寄存器AC里的值是0x800

Ø直接寻址：LOAD 0x800，直接寻址寄存器AC里的值是0x900

LOAD R1，直接寻址寄存器AC中的值是0x800

Ø间接寻址：LOAD 0x800，间接寻址，寄存器AC中的值是0x1000

Ø变址寻址：LOAD X，使用变址寻址，R1是变址寄存器，且R1中存放的值是1，则操作数有效地址是
X+1

Ø基址寻址：LOAD 0x800，R1，采用变址寻址和基址寻址，寄存器AC的值是0x500

寻址空间和什么相关？

- 直接寻址：地址位数决定寻址空间：12位地址寻址空间有 2^{12}
- 间接寻址：字长决定寻址空间：12位地址可寻址空间 2^{16}
- 变址&基址寻址：地址位数、字长决定寻址空间：指令中12位地址可寻址空间为： $2^{16}+2^{12}$

第六章

存储器类型

ROM (Read-Only Memory) :

Ø数据读取速度快、存储时间长，断了电还能存储数据

Ø例子：BIOS，CD-ROM，闪存

RAM (Random Access Memory) :

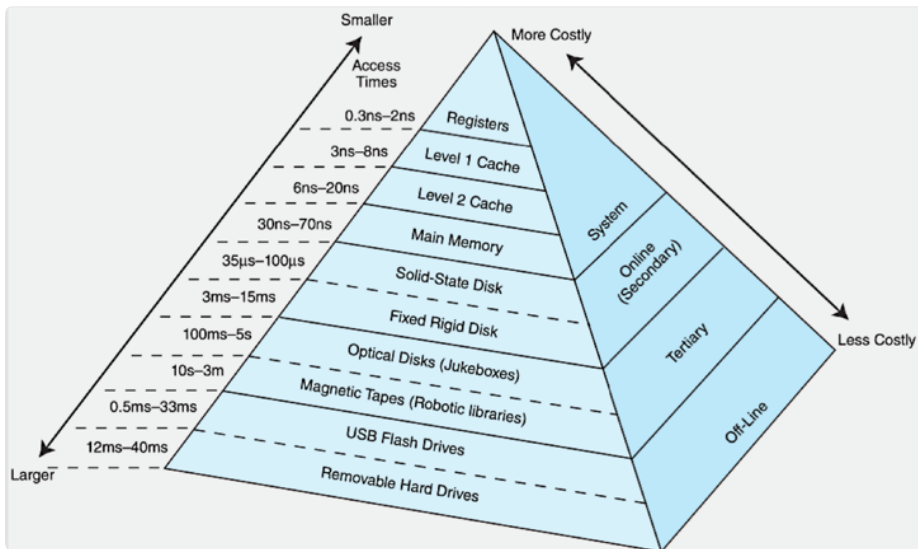
Ø可以对随机位置的数据进行读写，需要通电才能保存数据

Ø例子：主存

Ø按实现方式又分为两类：SRAM（静态随机访问存储器）和DRAM(动态随机访问存储器)

Ø通常采用DRAM实现主存储器，采用 SRAM实现高速缓存

存储器的层级化结构，ppt第八页的金字塔不会直接考，但是会涉及，但是很重要，有助于理解



存储器的层级化结构：方便利用时间局部性、空间局部性、顺序局部性

时间局部性(temporal locality):最近访问的数据在不远的将来会再次访问

空间局部性(spatial locality):即将访问的数据和最近访问的数据地址空间接近

顺序局部性(sequential locality):数据按顺序访问

衡量存储器性能的指标

命中 (hit):所需信息存储在该层存储结构中

Ø命中率 (hit rate):发生命中次数占总查询次数的比例

Ø命中时间 (hit time):在该层存储结构中进行一次查找的时间

失效 (miss):所需信息没有存储在该层存储结构中

Ø失效率 (miss rate):发生失效的次数占总查询次数的比例

Ø失效惩罚 (miss penalty):处理一次失效事件所需的所有时间开销，包括在下一层存储器中查询、将所需数据块传输到当前存储器中的时间开销

平均访问时间=命中率×命中时间+失效率×失效惩罚

Ø失效率+命中率=1

Ø命中时<<失效惩罚=(下一层平均访问时间+当前层次处理数据更新的时间)

有效访问时间(Effective Access Time, EAT): 描述一个**层次化存储系统**的性能

命中、失效、有效访问时间, ppt25页

例:假设一个两级层次化存储系统, 其中第一级访问时间为 10ns, 第二级访问时间为 100ns, 其中第一级访问命中率为90%

- 采用并行访问: $EAT = 0.9 \times 10ns + 0.1 \times 100ns = 19ns$

- 采用顺序访问:

$$EAT = 0.9 \times 10ns + 0.1 \times (10ns + 100ns) = 20ns$$

高速缓存基本原理

利用数据局部性, 用较小的高速访问存储器提高访问内存的性能。(现代计算机一般采用多级缓存)

高速缓存中的寻址依赖根据存储的内容, 因此也被称为按内容寻址存储器。

高速缓存中的寻址过程需要将主存地址映射为缓存中的地址, 因此也称为缓存映射。缓存通常采用和主存同样的存储单元(字节、字), 因此映射方法只取决于地址空间大小, 和内存寻址方法无关。

三种映射的形式

直接映射:

标记 (tag)+块号 (block)+块内偏移值 (offset)

全相联映射(Fully Associative Mapping) :

标记(tag)+块内偏移值(offset)

和直接映射相比: 缓存利用率高; 命中率更高; 标识位较长, 占用存储更多; 成本更高

组相联映射(Set Associative Mapping) :

标记 (tag)+组号 (set)+块内偏移值 (offset)

缓存的写策略

缓存的写策略体现在两类事件:

1. 脏块(dirty block): 载入到缓存中的数据块被修改, 致缓存中的块与主存中的数据不一致

写直通策略(Write-through): 写入缓存的同时将内容写入主存

写回策略(Write-back): 只写入缓存块, 当缓存块替换的时候写回主存

2. 写缺失(write miss): 写入的内存块不在缓存中

写分配策略(Write-allocate): 写入之前先载入对应的块, 然后采用脏块处理的策略进行写入

写不分配策略(No-write-Allocate): 直接写入主存

虚拟存储器解决的哪两个层次结构的什么问题?

1. (**主存**) 程序的可寻址地址空间大于主存的实际容量
2. (**硬盘**) 实际使用的内存空间大于主存的实际容量

虚拟存储器的核心思想:将一部分数据存储在硬盘上, 提高内存的存储能力
存储在硬盘上的部分称为页面文件(page file)

地址转换

1. 分页

虚拟地址分为两个字段

页编号: 该页在虚拟内存中的编号

页内偏移值: 该地址在虚拟内存中对应页内的偏移值

物理地址分为两个字段

页面编号: 该页在物理内存中对应的页面编号

页内偏移值: 该地址在对应页面中的偏移值

例: 假设一个按字节寻址的13位系统, 物理内存大小为4KB, 页面大小为1KB, 且进程1的页表如下, 请问下列虚拟地址对应的物理内存地址分别是多少:

- 0x1553 → 0x0553
- 0x0802 → 0x0002
- 0x1004

Page Table					
Page	Frame	Valid Bit	Page	Base 10	Base 16
0	-	0	0	0 - 1023	0 - 3FF
1	3	1	1	1024 - 2047	400 - 7FF
2	0	1	2	2048 - 3071	800 - BFF
3	-	0	3	3072 - 4095	C00 - FFF
4	-	0	4	4096 - 5119	1000 - 13FF
5	1	1	5	5120 - 6143	1400 - 17FF
6	2	1	6	6144 - 7167	1800 - 1BFF
7	-	0	7	7168 - 8191	1C00 - 1FFF

- 虚拟地址0x0802 = (0 1000 0000 0010) = (010 | 00 0000 0010)
- 页表中(010)=2对应表项有效位为1, 且页面编号为0
- 物理地址(00 | 00 0000 0010)=(0000 0000 0010)=0x0002

2. 分段

虚拟地址分为两个字段: 段编号和段内偏移值

对应的物理地址等于基地址加上偏移值

假设一个按字节寻址的系统采用13位虚拟地址，最多支持8个段，其中一个进程的段表如下，请问下列虚拟地址对应的物理内存地址分别是多少：

段号	基地址	有效范围	有效位
0	0000 0000 0000	100	1
1	-	-	0
2	0100 0000 0000	64	1
3	-	-	0
4	-	-	0
5	0101 0000 0000	512	1
6	-	-	0
7	-	-	0

- 0x1553
- 0x0852

- $0x1553 = (1\ 0101\ 0101\ 0011) = (101\ |\ 01\ 0101\ 0011)$ ，段号为 $(101)=5$ ，偏移值为 $(01\ 0101\ 0011)=0x153=339$
- 段表中5对应的有效位为1，对应的基地址为 $(01\ 10\ 0000\ 0000)$ ，有效范围为 $512 > 339$
- 基地址加上偏移值得到 $(01\ 11\ 0101\ 0011)=0x0753$

分页、分段

将每个进程的虚拟内存和系统物理内存划分为固定大小的数据单元，称为页(Page)，在物理内存中仅保存一部分进程的一部分页。虚拟内存页(简称虚页)需要经过地址转换为物理内存中的页。物理内存中的页称为页面(Frame/Page Frame)

分段管理允许一个进程拥有多个段(segment)，每个段的数据以一段连续的、长度可变的内存保存

TLB——转换后备缓冲器

为了减少访问页表的开销：使用转换后备缓冲器(Translation Lookaside Buffer, TLB)

TLB的本质就是将当前进程的页表放在缓存中，从而通过层次化存储结构提高访问页表的速度

TLB是一类特殊的缓存：保存的是虚拟页编号到物理页编号的映射

TLB通常采用相联缓存实现

ppt55页那个图!!!

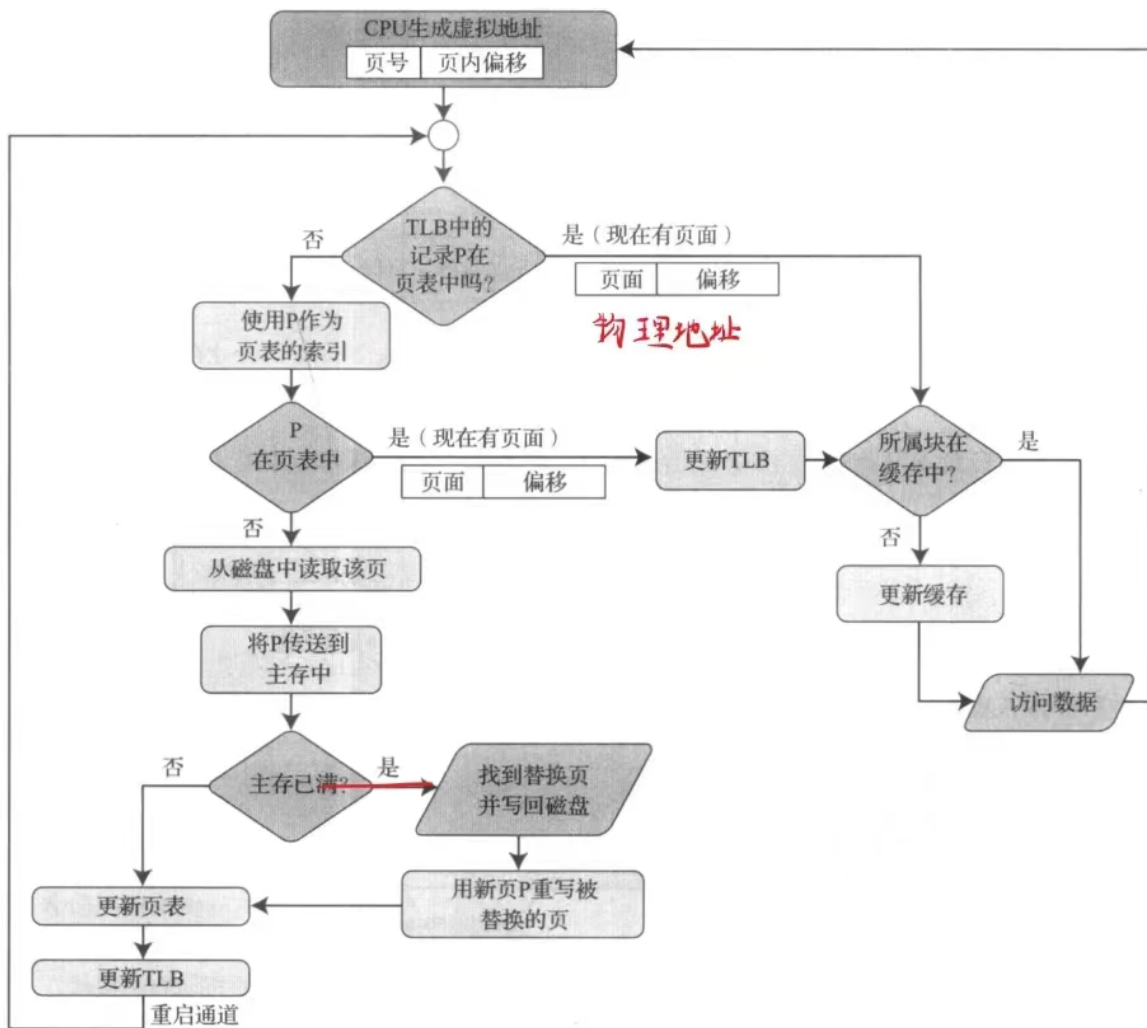


图 6-28 集中到一起：同时使用 TLB、页表、缓存和主存

1. 从虚拟地址中提取出页号和偏移值
2. 在TLB中查找虚拟页号对应的表项
3. 如果找到，则用对应的物理页号和偏移值生成物理地址
4. 如果未找到，需要访问内存中的页表
 - a. 如果该页在主存中，根据页表中的页面编号和偏移值生成物理地址，并更新TLB
 - b. 否则发生页面故障，触发中断，通常中断程序需要将页面加载到内存中，更新页表和TLB

第七章

独立磁盘冗余阵列——RAID

磁盘存储介质的数据可靠性受到环境的影响，发生数据错误的代价很大

所以：用多个便宜的普通磁盘组成可靠的存储系统 RAID

各个版本的RAID的特点

RAID-0：没有冗余备份，不提高可靠性，但是可以提高性能

RAID-1：也称为磁盘镜像、每份数据保存在两个独立磁盘上，100%的冗余存储

性能最高

RAID-2：是**理论**上组成方式，将硬盘分为两个集合:数据驱动器和校验驱动器，每个数据驱动器写入1位，用汉明码校验

RAID-3：也分为数据驱动器和校验驱动器，每个数据驱动器写入1位，只需要一个校验驱动器做奇偶校验

RAID-4：也是**理论**上的组合方式，对数据条带做奇偶校验，并行写入会导致奇偶校验驱动器竞争，造成性能瓶颈

RAID-5：为避免奇偶校验驱动器的竞争，不将奇偶校验放在同一个驱动器上，是应用最广泛的RAID系统

RAID-6：支持最多2个磁盘错误:采用两个校验码P和Q（P是奇偶校验码，Q是里德-所罗门校验码）

RAID-DP：可以恢复最多两个硬盘错误，采用两个奇偶校验码

RAID混合系统：采用多级RAID，每一级RAID采用不同的RAID类型（按照RAID组合方式命名，先组成RAID-1,然后组成RAID-0,命名为RAID-10）

I/O控制方法

程序控制I/O：有时称为轮询I/O，CPU不断地监视每个I/O端口是否有要发送的数据。是最简单的方式，但是频繁地轮询寄存器会让CPU在连续的“忙等待”中循环，直到开始服务某个IO请求，最适合用在某些专用系统上。

中断驱动I/O：多个I/O设备连到中断处理器，设备告诉CPU何时要有要发送的数据。CPU继续执行其他任务，直到请求服务的设备向CPU发送中断为止。在大多数中断驱动I/O的实现中，这种通信通过一个中间中断控制器进行通信。系统中的每一个外围设备都可以访问中断请求线，中断控制芯片对于每条中断控制线都有输入，但有优先级。

存储器映射I/O：CPU通过访问内存的方式直接访问I/O设备内存。I/O设备和主存储器共享相同的地划空间。因此，每一个I/O设备都有自己保留的存储块。I/O设备间的数据传输涉及向主存地址中移动，这个地址是映射到设备上的。这大大简化了系统设计，CPU不需要：该设备是否准备就绪，计算传输中的字节数，计算纠错码。

直接存储器存取（DMA）：使用专用芯片来编程（DMA处理器），CPU就不需要再执行冗长的I/O指令，可以独立地直接读写系统内存，而不需CPU介入处理。DMA控制器设备接口和CPU共享总线，DMA会利用由CPU使用的存储器周期来完成I/O操作这称为周期窃取。（如何共享:独占式共享、交替共享、周期窃取）

通道控制I/O：采用专用芯片，将多个I/O路径合并为I/O通道，由名为I/O处理器(IOP)的小CPU来控制，通道间有I/O桥通信。多为大型计算机系统使用。（合并的方法:对于慢速设备，采用多路复用通道(multiplexor);对于快速设备，采用选择器通道）

第九章

书364页的表9-1

ARM

表 9-1 RISC 和 CISC 的特征

x86

RISC	CISC
多个寄存器组，通常包含 256 个以上的寄存器	一个寄存器组，一般共包含 6 ~ 16 个寄存器
每条指令可包含 3 个寄存器操作数(例如，add R1, R2, R3)	每条指令可包含 1 个或 2 个寄存器操作数(例如，add R1, R2)
通过高效的片上寄存器窗口进行参数传递	通过低效的片外存储器进行参数传递
<u>单周期指令(除了 load 和 store 指令)</u>	多周期指令
硬连线控制	微程序控制
流水线条度高	流水线条度低
简单指令并且指令数较少	许多复杂指令
固定长度指令	变长指令
编译器设计复杂	微代码设计复杂
仅 load 和 store 指令可以访存	许多指令都可以访存
较少的寻址方式	许多寻址方式

一个指令一个电路
 电路设计比较简单, 功耗更低